

Jake Kim 김성주

I build cutting-edge financial software.

Product Engineer · Crypto-native Builder · AI-native Developer

jiovana.jake@gmail.com

github.com/howdyfrom2019

linkedin.com/in/sungjoo-kim-jake/

x.com/b_cryptojvke

t.me/b_cryptojvke

Yongin, Korea · Remote

Updated 2026-09-12

An AI-native product engineer who started in crypto and has built dApps, on-chain games, and an institutional trading terminal.

Right now I'm building OrderX, a multi-exchange crypto derivatives terminal. I use Claude and Codex as planner and reviewer, run the systems I design all the way to production, and like building big products with small teams.

Works

A trading terminal that puts options, futures, and spot across several exchanges on one screen. Piloted with a large multistrat fund; \$10B notional processed and \$1M in revenue during private beta. I led the frontend from initial setup and designed its 18 feature domains.

IMPACT

- \$10B notional processed and \$1M revenue during pilot and private beta. Pilot run with a large multistrat fund.
- Steady-state browser memory from 3.8GB down to about 1GB, so an all-day trading session no longer needs reloads.
- Grid engine replacement removed React re-renders during drag and resize; no UX degradation with many chart widgets on screen.
- AI workflow adoption let a small team finish the entire MVP feature flow. Localization automated through a Figma to i18n-agent pipeline.

PROJECTS

Browser memory optimization (3.8GB → 1GB) 2026.04 – 2026.08

Problem · Real-time streams, widget state, TradingView iframes, and query caches accumulated until Chrome sat at 3.8GB, slowing the UI and eventually killing the tab.

- Split orderbook and position-table updates down to row and symbol granularity to remove redundant re-renders and allocations.
- Replaced AG Grid with an in-house DataTable with row memoization, virtualization, and clean unsubscribe paths.
- Three-stage runtime recovery: partial resubscribe, then cache invalidation, then reload when returning from a background tab. Reloads are blocked while an order is being entered.

Translate-based in-house grid system 2026.06 – 2026.09

Problem · react-grid-layout recalculated layout and re-rendered React on every mousemove during resize, saturating the CPU.

- dnd-kit plus imperative DOM transform (translate) updates so React is out of the loop while dragging.
- Grid-to-pixel math isolated in pure functions and pinned with tests.
- Widget stretch and shareable page layouts with auto-generated OG images.

AI workflow adoption 2026.01 – 2026.09

Problem · A small team had to push MVP features across 18 domains at once, and AI output quality varied widely on a large codebase.

- Repository conventions, architecture, and domain rules fixed through context injection so any agent works to the same standard.
- Claude as planner, Codex as reviewer: design, implementation, and review separated, with plan/design documents and gap analysis as the audit trail.
- react-doctor cleanups run as a routine so debt from feature integration does not accumulate.

Figma MCP + i18n agent pipeline 2026.06 – 2026.09

Problem · Design changes and translated copy were applied by hand, so every release shipped with omissions and mismatches.

- Figma MCP reads the design source directly to apply component changes.
- An i18n agent finds missing keys, translates and registers them, and syncs with Lokalise. Localization runs on next-intl.

Chart library migration (lightweight-charts → TradingView) 2026.03 – 2026.09

Problem · The backend speaks gRPC-over-HTTP plus WebSocket rather than UDF, so the stock datafeed did not fit.

- Provider-pattern custom Datafeed. Position and execution overlays, live quote push, per-account overlays.
- Custom indicators (OI profile, visible-range profile, cross-exchange studies) and a range-preset toolbar.

AI chat orders and multi-exchange widgets 2025.12 – 2026.09

- UIContext v2 RFC and an @account-tag routing proposal so chat orders can express batches, triggers, and TP/SL; negotiated with the backend team.
- Spreader, Arb builder, Hedge simulation, RFQ, TWAP, One-tap buy. Documented the protocol limits of multi-account aggregation as a backend change request.

Frontend: Next.js 15, React 19, TypeScript, Bun, Tailwind v4, Radix UI, Jotai, TanStack Query, React Hook Form, Zod, dnd-kit, framer-motion, react-virtuoso · Realtime: Protocol Buffers (ts-proto) over HTTP + WebSocket · Chart: TradingView Charting Library v31, TitanCharts, ECharts · AI: Vercel AI SDK, assistant-ui, ElevenLabs · Tooling: Playwright, Bun test, Lokalise, PostHog, OpenTelemetry, AWS Amplify

The Waifu Sweeper team under Yield Guild Games (YGG). Started as Princess Sweeper, later rebranded to Waifu Sweeper / Playpsweeper. I owned the entire web client and contract stack and shipped design changes (reward multipliers, season leaderboards, Gold Mine mode, revive mode) on a weekly cadence.

IMPACT

- 19,939 payments through the shop contract with zero failures: 72,500 USDC, 25,000 USDT, 104.65 ETH, 50,616,446 YGG.
- Deployed to Abstract mainnet after an external audit (2025.12). 94 Hardhat tests cover purchases, the oracle, NFTs, and the session tracker.
- Upgradeable proxies allowed token-economy changes (reward multipliers, season leaderboards, Gold Mine, revive mode) without redeloys, supporting repurchase and retention.
- Owned the whole client and contract stack solo. Bot traffic blocked with Cloudflare Turnstile.

PROJECTS

Operating method: upgradeable proxies and context injection 2025.09 – 2026.07

Problem · The token economy (reward multipliers, repurchase incentives, seasonal events) changed weekly, and every touch of the payment contract put user funds at risk.

- ShopManager, NFT, and PriceFeed run behind UUPS proxies with a storage-layout review fixed into the deploy checklist. New capabilities such as the session tracker live in separate contracts rather than new proxy slots.
- Game rules, entities, and reward logic specified through context injection so in-game logic changes ship quickly and consistently.
- Result: 19,939 payments with no failures and weekly design releases.

AGW session-key game sessions 2026.05 – 2026.06

Problem · Repeated actions such as opening tiles, defeating monsters, finding items, and reviving had to be recorded on-chain without a wallet signature every time.

- A hook that creates, stores, and restores a 24-hour session key with a fee limit and per-contract, per-selector call policies.
- A WaifuGameSessionTracker contract that validates ordering with a per-player session id and action nonce.
- Passed the tracker address as an argument instead of adding storage to the already-deployed ShopManager proxy, and documented why.

NFT gacha and energy-pack shop 2025.09 – 2025.12

- ShopManager (UUPS): ETH/USDC/USDT/YGG payments with explicit per-token decimals, EIP-712 signature verification with nonce replay protection, slippage handling.
- PriceFeed (UUPS): Pyth ETH/USD feed, confidence checks, cent-denominated pack pricing.
- WaifuSweeperNFT (UUPS ERC721): batch minting up to 30, paginated tokensOfOwner to stay under gas limits.
- Client: bundles Pyth price updates into the transaction, NFT as a payment method, YGG discounts.

Live-ops tooling and seasons 2026.01 – 2026.07

- Season and Gold Mine leaderboards, reward dashboard, revive mode, referral-code validation.
- Maintenance page with a service schedule, per-feature and date-based flags.
- GA4 event design, Twitter Pixel, SEO/OG, BGM and sound effects, image preloading.

Frontend: React 19, TypeScript, Vite 7, Tailwind v4, Radix UI, Jotai, TanStack Query, Framer Motion, react-router 7 · Web3: Wagmi, Viem, Ethers v6, RainbowKit, Abstract Global Wallet (sessions) · Contracts: Solidity, Hardhat, OpenZeppelin Upgradeable v5, Pyth SDK, TypeChain · Infra: Vercel, Cloudflare Turnstile, GA4

Built the web and app platform for Gomble, the Web3 division of Korean game studio 111percent.

IMPACT

- Designed a dApp that kept wallet connection and transaction delivery stable through a TGE with more than 10,000 daily spike requests.

PROJECTS

Ad-reward app

- Native mobile app (React Native, Expo) and PWA.

NFT community site

- NFT verification, staking, and a request flow for commissioning NFT games.

Betting game contracts

- Upgradeable smart contracts using the proxy pattern.

\$GM token claim site

- Claim site for a token listed on Binance Alpha, Coinone, and Bitget.

Frontend: React, TypeScript, Vite, Bun, Vite-pwa, React Native, Expo · Web3: Viem, Ethers, Wagmi, WalletConnect, Hardhat, Solidity · Infra: AWS S3 + Route53, Vercel

GXC · xCBT Blockchain Engineer

2024.10 – 2025.04

Smart contract development and verification plus frontend integration for xCBT, a platform where players preview monthly blockchain game releases and compete for rewards.

IMPACT

- Built the social-verification mission platform that generated 10K+ USD.

PROJECTS

Social-verification mission platform

- Mission features.
- Wrote and updated NFT-related smart contracts.
- Client and back-office monorepo (pnpm, Turborepo).

Frontend: TypeScript, Next.js, Tailwind CSS, Framer Motion · Web3: Solidity, Hardhat, Viem, Thirdweb, Abstract Global Wallet

CatzeLabs · Yooldo Frontend Engineer

2023.04 – 2024.10

Frontend for Yooldo, a blockchain gaming platform that helps Web2 games transition to Web3, plus game onboarding support.

IMPACT

- Onboarded more than 900,000 users onto the platform.
- Contributed to up to \$150k monthly revenue through NFT purchase and verification UX.
- Helped secure investment from Consensus, the company behind MetaMask.
- Built eight or more in-house NFT mint pages grossing over 300M KRW.

PROJECTS

Yooldo platform frontend

- Platform development, operations, and on-chain event handling.
- Transaction-receipt analysis to verify in-game asset acquisition.
- Back-office features and a shared UI component library published to npm.

Frontend: TypeScript, Next.js, TanStack Query, Framer Motion, Storybook · Backend: Node.js, Nest.js, Prisma · Web3: Viem, Wagmi · Infra: Vercel, GitHub Actions

Built the in-house investment management tool for an accelerator focused on seed and pre-A tech startups.

IMPACT

- Automated investment-contract preparation that used to take more than three days.
- Consolidated portfolio tracking that had been scattered across teams.

PROJECTS

Startup investment management tool

- Improved deal-sourcing UX through interviews with investment associates.
- Time-series and conversion visualizations with recharts.

Frontend: TypeScript, React, Jest, recharts, styled-components · Infra: AWS S3 + Route53

Built and operated FOUND, a GPS-based lost-and-found reward app, in the Yongin area.

IMPACT

- 5,000 downloads on the Play Store.

PROJECTS

FOUND

- Found-item reporting flow (Android, Java).
- KakaoTalk chatbot for lost-item reports with a Flask/MongoDB webhook server.

Android: Java, Retrofit2, Glide · Backend: Python, Flask, MongoDB

Education & Credentials

Dankook University	B.S. Electrical & Electronic Engineering · 3.92/4.5	2016–2023
Engineer Information Processing	HRD Korea	2020
Patent · Bus fare payment method and server	KIPO	2020
Startup Idea Hackathon, Grand Prize	Gyeonggi CCEI	2018
Alchemy University Ethereum Bootcamp	Alchemy	2024